FOSSIL: A Software Tool for the Formal Synthesis of Lyapunov Functions and Barrier Certificates using Neural Networks

Alessandro Abate
University of Oxford
Oxford, United Kingdom
alessandro.abate@cs.ox.ac.uk

Daniele Ahmed
Amazon, Inc.
London, United Kingdom

Alec Edwards
University of Oxford
Oxford, United Kingdom
alec.edwards@cs.ox.ac.uk

Mirco Giacobbe
University of Oxford
Oxford, United Kingdom
mirco.giacobbe@cs.ox.ac.uk

Andrea Peruffo
University of Oxford
Oxford, United Kingdom
andrea.peruffo@cs.ox.ac.uk

ABSTRACT

This paper accompanies FOSSIL: a software tool for the synthesis of Lyapunov functions and of barrier certificates (or functions) for dynamical systems modelled as differential equations. Lyapunov functions are formal certificates for stability analysis, whereas barrier functions are formal certificates for the safety of dynamical models. FOSSIL is sound and automatic thanks to a counterexample-guided inductive synthesis loop. This method exploits the flexibility of candidate functions generated by training neural network templates, the formal assertions provided by a verifier (namely, an SMT solver), and finally new procedures to ease the exchange of information between the two mentioned components. We endow the tool with features of usability, scalability, and robustness—all of which are showcased on benchmarks.

CCS CONCEPTS

• **General and reference** → **Verification**; • **Computer systems organization** → **Embedded and cyber-physical systems**; • **Computing methodologies** → **Neural networks**.

KEYWORDS

Lyapunov functions, barrier certificates, neural networks, CEGIS, SAT modulo theory

ACM Reference Format:
Alessandro Abate, Daniele Ahmed, Alec Edwards, Mirco Giacobbe, and Andrea Peruffo. 2021. FOSSIL: A Software Tool for the Formal Synthesis of Lyapunov Functions and Barrier Certificates using Neural Networks. In *24th ACM International Conference on Hybrid Systems: Computation and Control (HSCC '21), May 19–21, 2021, Nashville, TN, USA.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3447928.3456646

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
HSCC '21, May 19–21, 2021, Nashville, TN, USA
© 2021 Association for Computing Machinery.
ACM ISBN 978-1-4503-8339-4/21/05…$15.00
https://doi.org/10.1145/3447928.3456646

1 INTRODUCTION

Context and Motivation for this Tool. We aim to tackle two fundamental problems in the analysis and verification of dynamical systems modelled as ordinary differential equations (ODE): stability analysis and safety verification. Both problems are core to control theory and entail relevant practical applications [5, 30]. Stability analysis concerns the convergence (in this work, asymptotic or practical) of trajectories around an equilibrium point of the ODE, whereas safety verification asserts whether trajectories generated from a set of initial conditions ever enter a given target set (deemed unsafe) - a model is said to be safe if this does not happen.

We are interested in sufficient conditions certifying the stability and the safety of ODE models: Lyapunov functions are key for stability analysis, whereas barrier certificates (henceforth, we shall refer to both as "functions") assert the safety of dynamical models. In other words, the existence (and practically, the construction) of (any) such function(s) ensures the stability (or safety) of the given model. We note that no control input (that is, external non-determinism) is considered in the dynamical models of this work.

We build recent theoretical contributions [1, 3, 4] into a codebase that implements state-of-the-art algorithms for the formal synthesis of Lyapunov and barrier functions and provide a new tool: FOSSIL[1].

Literature Background. The synthesis of Lyapunov and barrier functions has a long history in the control community.

Lyapunov function synthesis is classically approached either analytically (explicit, by hand) or using unsound numerical methods. Numerical methods for non-linear models include techniques that reduce the solution space to sum-of-squares (SOS) functions and relax the synthesis problem into a linear matrix inequality (LMI) program [20]. Relaxation-based methods typically assume polynomial templates and reformulate the problem as a semi-algebraic one [11, 31, 32] or as a linear program [23, 28, 29], and solve them using computer algebra systems (CAS) or SAT modulo theories (SMT) solvers. Alternatively, incremental methods construct candidates using genetic algorithms [36], fitting spatial samples or simulations [13]; then, they verify the candidates using CAS or SMT solvers. An approach that is relevant for the results of this work computes Lyapunov function by solving systems of linear inequalities that satisfy user-provided templates[28]. Related methods, which inspire this

[1] FOSSIL is available at https://github.com/oxford-oxcav/fossil

work, employ a counterexample-guided approach to synthesise control Lyapunov functions, using a combination of SMT solvers (for synthesis) and convex optimisation engines (for verification) [24–26]. Counterexample-guided approaches with SMT-based verification has been applied to polynomial templates and later extended to neural networks [1, 3]; a similar setup is considered in [7], however with different architectural details. Alternative approaches for Lyapunov neural networks use simulations and numerical optimisation, all of which are unsound [27].

For polynomial models, barrier functions can be generated by solving sum-of-squares (SOS) and semi-definite programming problems [17, 21, 33]. To increase scalability and to enhance expressiveness, a few different barrier formats have been considered [15, 34, 37]. Some optimisation-based approaches provide formal correctness guarantees by verifying their result via CAS a posteriori [8, 16]. Cognate work presents a method to compute barrier functions using neural networks (NNs) and to verify their correctness a posteriori [38]: as such, it does not generate counterexamples within an inductive loop, as in this work. Recent contributions have employed NNs for the inductive synthesis of Lyapunov functions and barrier certificates via CEGIS [1, 3, 4]. This work uses similar template-free techniques based on NN, improving the communication between learner and verifier in the CEGIS loop and endowing the approach with scalability and numerical robustness features.

Contributions. We present FOSSIL, a new software tool for the automated formal synthesis of Lyapunov and barrier functions. This tool implements an approach that efficiently exploits machine learning, whilst guaranteeing formal proofs of correctness via SMT. We leverage a counterexample-guided inductive synthesis (CEGIS) engine [35], namely a technique that casts a synthesis problem as an inductive loop between a *learner* and a *verifier* (cf. Fig. 1). We devise two tailored learners to synthesise Lyapunov and barrier functions respectively, which train a neural network to fit, over a finite set of samples, the requirements for a candidate (Lyapunov or barrier) function, which are expressed through a loss function; the implemented verifiers through an SMT solver either formally prove the validity of the candidate function, or provide counterexamples: the counterexamples indicate where the conditions are violated, and are passed back to the learner for further training. In addition to implementing this novel technique, FOSSIL provides the following three contributions. First, it enriches it with a programming interface: developers will find a programming ground to extend FOSSIL to their needs. Our implementation provides an abstract interface to exploit our components or add new ones by conforming to the API. Second, it improves its performance due to an enhanced communication between learner and verifier. Third, FOSSIL is tested and challenged on a number of benchmarks, and is shown to have numerical robustness features.

Organisation. The remainder of the paper is organised as follows: Section 2 provides background on the synthesis architecture (CEGIS) underpinning FOSSIL. Section 3 discusses the usability and user interface of the tool. Section 4 discussed the structure of FOSSIL: in particular, Sec. 4.1 introduces the neural training technique, whereas Sec. 4.3 illustrates the SMT-based verification. Other parts detail two modules that enhance the communication between the learner and the verifier. Section 5 discusses the tool functionality

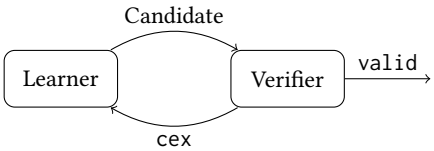

Figure 1: CEGIS loop for formal synthesis (cex represents counterexamples).

via benchmarks and possible threats to generality, with studies challenging scalability and robustness, and various correctness tests. Section 6 summarises the contributions and delineates future work.

2 THEORETICAL UNDERPINNINGS

Stability and Safety of Dynamical Models. This work aims at studying two major properties of dynamical models: *stability* and *safety*. We briefly introduce their definition and discuss their features, as they are closely related. We consider autonomous n-dimensional systems of non-linear ODEs

$$\dot{x} = f(x), \quad x \in \mathbb{R}^n, \tag{1}$$

where $f : \mathbb{R}^n \to \mathbb{R}^n$ is a continuous vector field, and $x(t)$ denotes the value of the state x at time t, where $t \in \mathbb{R}$.

For stability analysis, we focus on an equilibrium point x_e, $f(x_e) = 0$, and a domain of interest $\mathcal{D} \subseteq \mathbb{R}^n$ containing x_e. Stability analysis determines whether a dynamical system, initialised in $x(0) = x_0 \in \mathcal{D}$, never escapes a domain of interest around an equilibrium point x_e and, possibly, converges asymptotically towards that point.

For safety verification instead, we assume that the initial point $x(0) = x_0 \in X_0$, where X_0 represents the set of initial states, and furthermore consider an unsafe set X_u - we assume that both are subsets of a domain $\mathcal{D} \subseteq \mathbb{R}^n$. The safety verification problem concerns checking whether any trajectory of the model originating from X_0 reaches the unsafe region X_u.

Stability Analysis and Safety Verification. The notion of stability is connected to the physical concept of energy of the system, and of its evolution over time. To analyse the energy evolution over time we introduce a real-valued function C over the state space $\mathbb{R}^n$, and evaluate how $C(x)$ changes along the flow of an ODE using an operation called *Lie derivative*. More formally, consider a continuously differentiable scalar function $C : \mathbb{R}^n \to \mathbb{R}$. The Lie derivative of C with respect to a vector field f is defined as

$$\dot{C}(x) = \nabla C(x) \cdot f(x) = \sum_{i=1}^{n} \frac{\partial C}{\partial x_i} \frac{dx_i}{dt} = \sum_{i=1}^{n} \frac{\partial C}{\partial x_i} f_i(x). \tag{2}$$

Intuitively, this derivative denotes the rate of change of function C along model trajectories.

Let us now introduce the notion of Lyapunov functions, which are formal certificates for the *asymptotic stability* of ODEs [30]. A Lyapunov function maps system states x into energy-like values that are bounded from below at the equilibrium x_e, and it additionally decreases over time along model trajectories. If one such function exists, then the system is asymptotically stable within $\mathcal{D}$. This is a real-valued function $C : \mathbb{R}^n \to \mathbb{R}$ such that $C(x_e) = 0$ and,

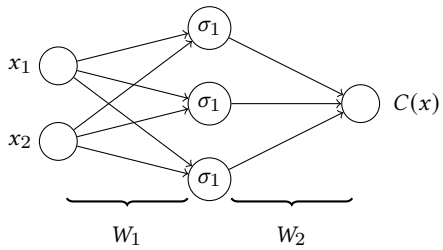

Figure 2: A feed-forward neural network with one hidden layer.

for all states $x \in \mathcal{D}$ other than x_e, it satisfies the two conditions

$$\dot{C}(x) = \nabla C(x) \cdot f(x) < 0, \qquad C(x) > 0. \tag{3}$$

We can relax the first requirement to hold everywhere in $\mathcal{D}$, except for a δ-neighbourhood of x_e defined as $\{x \in \mathcal{D} : \|x - x_e\| \leq \delta\}$: this leads to establishing *δ-practical* (or Lagrange) stability [5].

Let us now discuss the notion of barrier functions for safety verification [21], which offer sufficient conditions asserting the safety of the model, namely when no trajectory starting from X_0 enters the unsafe region X_u. More formally, let the model in (1) and the sets $\mathcal{D}$, X_0 and X_u be given. Suppose there exists a function $C : X \to \mathbb{R}$ that is differentiable with respect to its argument and satisfies the following conditions:

$$C(x) \leq 0 \; \forall x \in X_0, \quad C(x) > 0 \; \forall x \in X_u, \quad \dot{C}(x) < 0 \; \forall x \in \mathcal{D} \text{ s.t. } C(x) = 0, \tag{4}$$

then the safety of the model is guaranteed. Indeed, consider a trajectory $x(t)$ starting in $x_0 \in X_0$ and the evolution of $C(x(t))$ along this trajectory. Whilst the first of the three conditions guarantees that $C(x_0) < 0$, the last condition asserts that the value of $C(x(t))$ along a trajectory $x(t)$ cannot increase, hence cannot become positive. Such a trajectory $x(t)$, starting from the initial set X_0, cannot enter the set X_u, where $C(x) > 0$ (second condition), thus ensuring the safety of the model.

Synthesis of Lyapunov and Barrier Functions via CEGIS. We recall the CEGIS procedure for the construction of Lyapunov functions in the form of feed-forward neural networks (NN, Fig. 2). This method exploits efficient machine learning algorithms, while guaranteeing formal soundness via SMT solvers.

The CEGIS procedure is iterative, as follows. Firstly a numerical *learner* trains a candidate to satisfy the conditions in either (3) or (4) over a set S of samples, then a formal *verifier* confirms or falsifies whether the conditions are satisfied over the whole dense domain $\mathcal{D}$. If the verifier falsifies the candidate, one or more counterexamples are added to the samples set and the network is retrained. The procedure repeats in a loop until possibly the verifier confirms the candidate.

Learner. The learner trains a NN with multiple layers and activations functions that are polynomial of any degree, or non-linear expressions (e.g., a tanh function). We denote the overall structure of the NN as $\mathcal{N}(\theta)$, where θ represents a user-selected set of NN parameters, depending on its width and depth of the architecture. Let us consider a network with an input layer composed of n neurons, corresponding to the dimension of the dynamical model in (1), followed by k hidden layers with respectively $h_1, \ldots, h_k$ neurons, and finally followed by one output neuron – recall that both Lyapunov and barrier functions are scalar functions. Nodes of adjacent layers are connected via a matrix $W_i \in \mathbb{R}^{h_i \times h_{i-1}}$, which contains the weights from the $(i-1)$-th to the i-th hidden layer. Finally, matrix $W_{k+1} \in \mathbb{R}^{1 \times h_k}$ contains the weights from the k-th layer to the last neuron. Every i-th hidden layer comes with a non-linear activation function $\sigma_i : \mathbb{R} \to \mathbb{R}$, whereas the output neuron is free of activation and bias. The valuation of output and hidden layers is given by

$$z_{k+1} = W_{k+1} \cdot z_k, \quad z_i = \sigma_i(W_i \cdot z_{i-1} + b_i), \quad i = 1, \ldots, k, \tag{5}$$

where b_i is the i-th bias vector, and each σ_i is applied element-wise to its h_i-dimensional argument. Upon assigning the vector z_0 to the input layer, we evaluate (5) layer by layer, resulting in the function

$$C(z_0) = z_{k+1}. \tag{6}$$

Figure 2 depicts a neural network of this kind with $k = 1$, $n = 2$, $h_1 = 3$, and the weight matrices W_1 and W_2.

Conditions (3) and (4) can be cast into a multi-objective training goal that is non-trivial for neural architectures. The choice of a tailored loss function (see Sec. 4.1) is thus crucial for the efficacy of the approach.

Verifier. The NN is translated into a symbolic candidate C and passed to the verification engine. Deciding whether a neural network candidate C is a proper Lyapunov/barrier function for the model in (1) amounts to deciding second-order logical formulae, call them φ_1 or φ_2, which correspond to the requirements in (3) and (4), respectively. Moreover for stability analysis, φ_1 can be checked over an open domain around the equilibrium x_e for asymptotic stability, or over a domain that excludes a δ-neighbourhood of x_e for practical stability.

The soundness of our tool relies on the correctness of the underlying SMT solver: we use two solvers (Z3 [9] and dReal [10]) within CEGIS to provide correct outcomes based on NN candidates. For verifying systems of polynomial ODEs together with neural networks with polynomial activation functions we provide the option of using the SMT solver Z3, exploiting an efficient decision procedure for non-linear real arithmetic [12]. For general models which may include trigonometric or exponential expressions (for instance, the models considered in Section 5) we use dReal. However, dReal is a δ-complete solver which guarantees that the result of our CEGIS loop is is correct when determining the candidate is valid, however may produce false counterexamples within a δ-error bound. This might lead to many, possibly infinite, iterations of the CEGIS loop for some choices of parameters, however this does not compromise the correctness of the final result if the procedure terminates. Within our setting, the δ precision may cause issues when checking a Lyapunov candidate around the point x_e or a barrier candidate over the set $C(x) = 0$. For this reason, we employ dReal to study *practical stability*, hence excluding from the domain of a neighbourhood around x_e, and we relax the barrier constraint to $|C(x)| \leq \tau_v$, which conservatively strengthens it. This preserves

correctness of the final result, while improving convergence of the overall loop.

3 FOSSIL: USABILITY AND USER INTERFACE

FOSSIL is implemented in Python: the primary interface for a user takes the form of a Jupyter Notebook [14], which serves as an interactive 'playground' for users to freely access the tool. We anticipate varying user proficiency and use cases by providing two notebooks. The first allows users to define a desired dynamical system as symbolic variables alongside relevant domains for either Lyapunov or barrier functions. Users are then able to synthesise a corresponding function using default parameters for CEGIS. The second notebook details how the user can modify all of the parameters in the CEGIS procedure and facilitates running the tool with these custom parameters. For example, it provides control over the structure of template neural network for the learner, denoted by $\mathcal{N}(\theta)$. This includes the number of layers, of neurons in each hidden layer, and the corresponding activation functions. Users may specify the SMT solver of choice.

The input from the user is then passed to the *primer* component (cf. Fig. 3), which acts as liaison between the user and the CEGIS block by preparing the input for the CEGIS engine (cf. Fig. 4). We will now detail how the interface is used, as well as how this input is handled by the primer, before it is passed to CEGIS. This includes generation of the set S, a set of M data points encompassing the domain $\mathcal{D}$, which is utilised by the learner for training. Note that users are able to bypass the Jupyter Notebook and interface directly with the primer, if desired.

Lyapunov Functions. There are two inputs for Lyapunov synthesis: a dynamical system f and a domain $\mathcal{D}$, the latter of which is defined as a hypersphere of radius r around an equilibrium point. For practical stability, it is possible to exclude a spherical region around the equilibrium point from this, resulting in a domain shaped as a hyper-torus.

We provide the following demonstrative example to illustrate the usage of the notebook. Consider the following two-dimensional

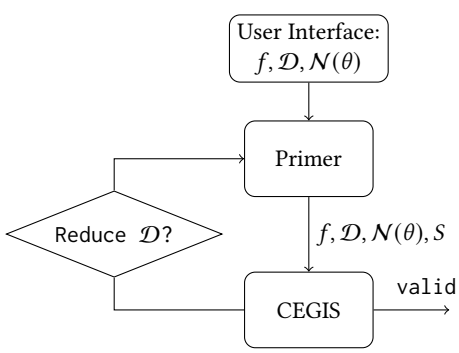

Figure 3: External interface of FOSSIL. Here, f represents the continuous vector field provided by the user, $\mathcal{D}$ is the domain of interest, $\mathcal{N}(\theta)$ denotes the nerual network structure and S is the set of M data points used by the learner for training.

dynamical system, which we denote as benchmark $\texttt{nonpoly}_0$:

$$\begin{cases} \dot{x} = -x + xy, \\ \dot{y} = -y, \end{cases} \tag{7}$$

We define the domain as a circle of radius 10, $\mathcal{D} = \{x^2 + y^2 \leq 100\}$. These can be instantiated in the notebook as follows:

```
x0, x1 = initialise_states(2)
f = [
  -x0 + x0 * x1,
  -x1
]
radius = 10
```

Once this is passed to the primer component, the first stage of processing involves finding the equilibrium points of the provided dynamical system f. This is achieved by solving $f(x) = 0$ symbolically using the Python symbolic library *sympy* [19]. Each equilibrium point x_e found by this process is then locally checked for stability by the following procedure: the function f is symbolically differentiated to obtain the Jacobian $J(x)$ [30] (namely, the matrix of partial derivatives), which is evaluated at the equilibrium x_e. Stability is then ascertained through *numerical computation* of the eigenvalues of the corresponding Jacobian matrix $J(x_e)$.

The next steps depend on the number of stable equilibria found:

- *No stable equilibria found*: Lyapunov analysis cannot be performed. The user is given the opportunity to provide a candidate equilibrium, in case there has been an error in the eigenvalues computation. This is checked and analysis continues.
- *Single stable equilibrium found*: The tool proceeds automatically with this as the equilibrium.
- *Multiple stable equilibria found*: The user is presented with the stable equilibria and may choose which one they wish to perform Lyapunov analysis on.

Once an equilibrium point x_e has been selected, the dynamical system $f(x)$ is translated to $f(x+x_e)$ to ensure that the equilibrium point is located at the origin. The primer then passes this to the CEGIS engine, alongside the corresponding domain and relevant parameters (the default ones, unless specified by the user).

For the example $\texttt{nonpoly}_0$, there is only a single equilibrium point at the origin, $x_e = (0, 0)$. The resulting Jacobian evaluated at this equilibrium point is

$$J(x_e) = \begin{pmatrix} -1 & 0 \\ 0 & -1 \end{pmatrix}.$$

As $J(x_e)$ has a repeated eigenvalue of -1, the primer module will determine there is a single stable equilibrium point and proceed.

If the CEGIS procedure ever fails to synthesise a valid Lyapunov function for a domain $\mathcal{D}$, FOSSIL provides features of interactive/adaptive domain refinement. The tool determines the data point in the set S (cf. Fig 4) that is closest to the origin and that does not satisfy (3). The distance between this point and the origin is then offered to the user as a new radius for the updated hypersphere $\mathcal{D}$. This adaptive domain management is an optional feature that seeks to assist stability analysis in scenarios with pathological dynamics.

Barrier Functions. In addition to a dynamical system f, the synthesis of a barrier function requires the declaration of three domains: $\mathcal{D}, X_0$ and X_u. For automated synthesis of barrier functions and in the interest of robustness, these domains are limited to be hyper-spheres or hyper-rectangles, however users are able to define arbitrary domains and pass these to the tool. For our purposes, hyper-spheres are characterised by their centre and radius, while hyper-rectangles are characterised by their lower and upper bounds. Suppose we wish to input the following domains: $\mathcal{D} = \{x_0 \geq -2, x_0 \leq 5, x_1 \geq -5, x_1 \leq 2\}, X_0 = \{(x_0+0.5)^2+(x_1-0.5)^2 \leq 0.16\}, X_u = \{(x_0-0.7)^2+(x_1+0.7)^2 \leq 0.09\}$. Note that this is equivalent to the domain in benchmark $\texttt{barr}_2$ in Section 5.1. This can be instantiated as follows:

```
D   = Rectangle([-2, -5], [5, 2])
X_0 = Sphere([-0.5, 0.5], 0.4)
X_u = Sphere([0.7, -0.7], 0.3)
```

These domains, alongside suitable user-defined dynamics f, are passed from the interface to the primer, which generates appropriate methods for evaluating the domains and data within them before passing these to CEGIS.

We also provide a third Jupyter Notebook containing case studies for synthesis of Lyapunov and Barrier functions.

4 FOSSIL: TOOL STRUCTURE AND COMPONENTS

The learner and the verifier belong to two very different domains: the first manipulates numerical values while training a NN, whereas the second handles logical formulae. The communication between these two parts is thus a key feature to guarantee an efficient and reliable operation of the overall loop. To this end, we introduce two additional components: a translator, which converts the NN into a symbolic expression of the candidate C, and a consolidator, which augments the counterexample generated by the verifier to support and enhance the following synthesis round.

4.1 Learner

The CEGIS loop starts with the learner: it instantiates a neural network using the hyper-parameter θ and trains it over the M samples in the set S until a stopping criterion is met. The class of candidates comprises multi-layered, feed-forward NNs equipped with a wide choice of activation functions: polynomials (e.g. identity, square, quartic) and $\tanh$. In contrast to most learning applications, polynomial activations return a readily interpretable candidate.

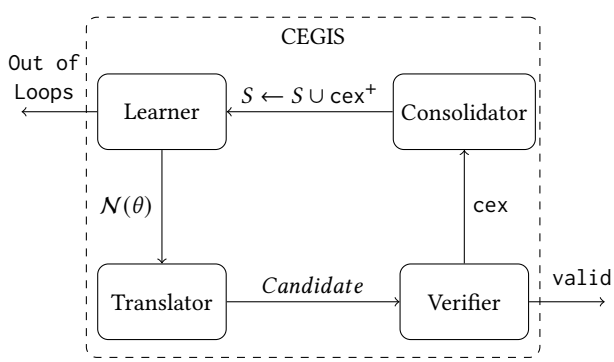

Figure 4: CEGIS modular architecture within FOSSIL as part of Figure 3, is an extension of Figure 1.

Training. The learning process updates the NN parameters to improve the satisfaction of the conditions in (3) or (4), following the gradient of a loss function that is a proxy for (3) or (4). For brevity, we now illustrate the choice of the loss for the synthesis of a Lyapunov function (adaptations for barrier synthesis are at the end of this paragraph). The training minimises a loss comprising two terms, L_p and L_d, guiding towards a positive C and a negative $\dot{C}$, respectively. Namely

$$L = L_p + L_d = \frac{1}{N}\sum_{s_i \in S}\Big(\mathrm{LR}_\alpha(-C(s_i)-\tau) + \mathrm{LR}_\alpha(\dot{C}(s_i)-\tau)\Big), \quad (8)$$

where s_i, $i = 1, \ldots, M$ are the samples taken from the set S. By LR_α we denote the Leaky ReLU function with parameter $\alpha > 0$, as

$$\mathrm{LR}_\alpha(x) = \mathrm{ReLU}(x) - \alpha\,\mathrm{ReLU}(-x) = \begin{cases} x & \text{if } x \geq 0, \\ -\alpha x & \text{otherwise.} \end{cases} \quad (9)$$

The constant τ is an (positive) offset, added to improve the numerical stability of the training. The loss function in (8) embeds two objectives: the *ReLU* part penalises samples with incorrect values, e.g. a sample s yielding a negative $C(s)$, whilst the *Leaky* part rewards samples that evaluate to a correct value, e.g. a positive $C(s)$. Incorrect samples account for the major contribution to the loss function, leading the NN to correct those first. Then, the network finds a direction of improvement following the *leaky* portion of the loss function.

For the case of barrier functions we set three components L_0, L_u and L_d for the conditions on the initial and unsafe sets, and for the derivative, respectively, using Leaky ReLU functions. Terms L_0 and L_u replicate L_p, where the summation is computed over the respective sets S_0 and S_u. The derivative condition represents a challenge for the training: $C(x) = 0$ can be a measure-zero set, thus it is highly unlikely that a single sample s will satisfy $C(s) = 0$. We thus relax this last condition and consider a belt-shaped set $\mathcal{B}$ around $C(s) = 0$, namely $\mathcal{B} = -\beta_1 \leq C(x) \leq \beta_2$, which depends on the (positive) hyper-parameters β_1, β_2. The loss term L_d computes the summation over samples contained in $\mathcal{B}$.

Multi-layer Networks. We have experienced that multi-layer NN provide numerical robustness to our method. In view of our choice of polynomial activations, the order of the candidate increases with the network depth, along with the verification complexity. To overcome this challenge, our procedure leverages a multi-layer architecture whilst maintaining a low-order polynomial by interchanging identity and polynomial activations over adjacent layers. We have observed that the use of identity activations, particularly in the output layer, positively affect the training: they provide robustness without increasing the order of the network with new polynomial terms.

Exploiting polynomial activations. A canonical template for Lyapunov function is a SOS polynomial, in view of its (semi)definite positiveness [20]. A candidate that originates from such template guarantees the second condition in (3) to be fulfilled. In our setting,

this translates into a faster training round (the network only follows the gradient $\dot{C}$) and a lighter verification check (again, only the first condition should be certified).

We implement a feature matching this observation. The expression of a candidate C depends both on the weight matrices W_i and on the activation functions. By choosing square (or quadratic, etc.) activation functions, we guarantee that all elements of each layer z_i are non-negative. As per Eq. (6), the final value of C is a linear combination of the elements of z_k (non-negative using even-polynomial activations) with coefficients contained in the last network layer W_{k+1}. If we ensure the last-layer elements to be positive, we automatically guarantee (semi)definite positiveness of the candidate C. We do so with the LLO feature, which can be enabled by the user and sets the last-layer elements to be equal to one.

4.2 Translator

The interface between the numerical learner and the formal verifier is managed by the translator: it translates the trained neural network into a symbolic candidate certificate C (see Fig. 4).

Let us recall the computation of the numerical network values in Eq. (6): we adapt a similar method to compute the symbolic evaluation of the network. Upon assigning the symbolic vector $z_0 = [x_1 \dots x_n]$ to the input layer, the translator evaluates equation (5) layer by layer, resulting in the function

$$C(x) = z_{k+1}, \qquad z_0 = [x_1 \dots x_n]. \tag{10}$$

We evaluate the expression of $\dot{C}(x) = \nabla C(x) \cdot f(x)$ directly from the structure of the NN, thus avoiding a symbolic differentiation of $C(x)$. To compute the gradient of $C(x)$ over x we use the chain rule

$$\nabla C(x) = \frac{\partial C}{\partial x} = \frac{\partial z_{k+1}}{\partial z_0} = \prod_{i=1}^{k+1} \frac{\partial z_i}{\partial z_{i-1}}. \tag{11}$$

The first k factors result in

$$\frac{\partial z_i}{\partial z_{i-1}} = \frac{\partial \sigma_i(W_i z_{i-1} + b_i)}{\partial z_{i-1}} = \mathrm{diag}[\sigma_i'(W_i z_{i-1} + b_i)] \cdot W_i, \tag{12}$$

where σ_i' is the full derivative of function σ_i and $\mathrm{diag}[v]$ represents a diagonal matrix whose entries are the elements of vector v. For the last factor instead it simply holds

$$\frac{\partial z_{k+1}}{\partial z_k} = W_{k+1}. \tag{13}$$

Finally, the gradient results in

$$\nabla V(x) = W_{k+1} \cdot \prod_{i=1}^{k} \mathrm{diag}[\sigma_i'(W_i z_{i-1} + b_i)] \cdot W_i. \tag{14}$$

Along with the symbolic expression of $f(x)$ provided by the user, we ultimately obtain $\dot{C}(x) = \nabla C(x) \cdot f(x)$.

In the Lyapunov function instance, we must ensure $C(x_e) = 0$. Without loss of generality, let us assume $x_e = 0$: we guarantee $C(0) = 0$ by setting zero bias for all layers and choosing activation functions that evaluate to zero with a zero input.

The efficiency of SMT solvers depends also upon the numerical expressions of the formulae to be verified. Hard-to-handle candidates may arise for several reasons during the synthesis, e.g. in the presence of coefficients with both large and small magnitude, or coefficients with an unreasonable number of digits, which are indeed rather frequently generated by NN-based numerical algorithms. These candidates might slow the verification step, and thus the whole procedure. Ultimately, the coefficients of the symbolic candidate depend on the elements of matrices W_i, as per (10). During the evaluation of $C(x)$ and $\dot{C}(x)$, we thus apply a rounding operation to the weight matrices,

$$\hat{W_i} = \mathrm{round}(W_i, d),$$

where d is the desired number of digits.

4.3 Verifier

At the outset the primer instantiates the verifier with the symbolic domain $\mathcal{D}$ along with the vector field f. The verifier then integrates them with the candidate $C(x)$, to check the validity of formulae φ_1 or φ_2 corresponding to (3) or (4). At each CEGIS loop, the verifier receives a regularised candidate $C(x)$ from the translator, in the form of a symbolic formula. Every new candidate $C(x)$ requires to be certified: practically speaking, the SMT-based verifier aims at finding states that violate the conditions in (3) and (4) over the continuous domain $\mathcal{D}$. If no such examples are found, $C(x)$ is a valid certificate and the CEGIS loop terminates. To this end, we express the negation of such requirements φ_1 or φ_2, and formulate a nonlinear constrained problem as

$$\exists x \colon x \in \mathcal{D} \wedge (C(x) \leq 0 \vee \dot{C}(x) \geq 0), \tag{15}$$

for a Lyapunov function, or, for a barrier certificate,

$$\exists x \colon (x \in X_0 \wedge C(x) > 0) \vee (x \in X_u \wedge C(x) \leq 0) \\ \vee (C(x) = 0 \wedge \dot{C}(x) \geq 0). \tag{16}$$

which determine whether a counterexample exists. If so, the verifier passes the counterexample to the consolidator, which enriches it for further use of the learner.

The effectiveness of the tool is underpinned by rapid operations and exchanges between the components. Particularly for the barrier functions instance, the constraint $\eta_d = (C(x) = 0 \wedge \dot{C}(x) \geq 0)$ often reduces the fluidity of the loop, since the formula is defined over the thin region characterised by $C(x) = 0$. For this reason, we prioritise constraints $\eta_0 = (x \in X_0 \wedge C(x) > 0)$ and $\eta_u = (x \in X_u \wedge C(x) \leq 0)$ that is, if either clauses is satisfied, i.e. a counterexample is found for at least one of them, the verifier omits testing η_d whilst the obtained counterexamples are passed to the learner. The constraint η_d is thus checked solely if η_0 and η_u are both deemed to be unsatisfiable. Whenever this occurs and the verification of η_d times out, the solver searches for a solution of a relaxed constraint $(|C(x)| \leq \tau_v \wedge \dot{C}(x) \geq 0)$, similarly to the improved learning conditions with the relaxation introduced with $\mathcal{B}$ discussed in Sec. 4.1. Whilst this constraint is arguably easier to solve in general, it may generate spurious counterexamples, namely a sample $\bar{s}$ that satisfy the relaxed constraint, but such that $C(\bar{s}) \neq 0$. The generation of these samples does not harm the correctness of the overall procedure, and indeed are shown to improve the robustness of the next candidate—this of course comes at the cost of increasing the number of iterations.

4.4 Consolidator

The verification block returns a counterexample (denoted cex in Fig. 4), namely a (single) instance satisfying (15) or (16). However, a lone sample might not always provide insightful information for the learner to process. Naïvely asking the SMT solver to generate more than one counterexample can be in general expensive, as it must be done either sequentially or by augmenting the verification formula. Specifically, the verifier solves (15) or (16) to find a first counter-example $\bar{s}$; then, to find any additional sample, we include the statement $(x \neq \bar{s})$ and solve again for the resulting formula.

We thus devise an heuristic approach to increase the information generated by the verifier: first we sample around the counterexample and then we follow the direction that maximises the violation of the certificate conditions. To this end, we randomly generate a *cloud* of points around $\bar{s}$: in view of the continuity of the candidate function C, samples around a counterexample are also likely to invalidate the certificate conditions. Secondly, starting from $\bar{s}$, we compute the gradient of C (or of $\dot{C}$) and follow the direction that maximises the violation of the certificate constraints. In the barrier function instance, we follow the maximisation of C (resp. $\dot{C}$) when considering $\bar{s} \in X_0$ ($\bar{s} \in \mathcal{B}$), and minimisation of C when $\bar{s} \in X_u$. In the Lyapunov instance, we follow the minimisation (maximisation) of C ($\dot{C}$). This gradient computation is fast thanks to an automatic differentiation feature, so it does not impact the overall computational time. The consolidator then aggregates the original counterexample and the newly generated points (denoted cex^+ in Fig.4) with the sample set S for a new synthesis round by the learner.

5 FUNCTIONALITY

We test FOSSIL over a suite of non-linear examples taken from related work on Lyapunov [2, 11, 22, 28] and barrier function synthesis [6, 18, 21, 37, 38] and present new benchmarks. The case studies present non-standard features (non-convex domains, hybrid vector fields) that are not available for basic users through the interface. Nevertheless, advanced users are free to build on the existing case studies to externally code bespoke problem settings. This tool builds upon previous contributions: in [1] and [4] we compare our approach for the synthesis of Lyapunov and barrier functions, respectively, against SOSTOOLS [20] and state-of-the-art methods [7, 28, 38]. Our approach proves to be (up to 100 times) faster whilst using a smaller (up to 10^3 times) sample set. The present contribution, besides formally presenting the tool, focuses on testing different benchmarks and on a robustness analysis.

5.1 Tool Evaluation via Benchmarks

The case studies may be divided into three categories. The first set of benchmarks includes four polynomial models [2, 11] that admit a global, non-polynomial Lyapunov function. We challenge our procedure to find a local polynomial Lyapunov function over a bounded domain. We emphasise that finding local Lyapunov functions is still useful in many applications, and perhaps serves as a measure of the tool's efficacy. We begin our tests with $\mathsf{nonpoly}_0$ [2], which is 2-dimensional and polynomial of order 2. The Lyapunov function synthesised for this benchmark is depicted in Fig. 5. The complexity of the benchmarks increases with $\mathsf{nonpoly}_1$ a

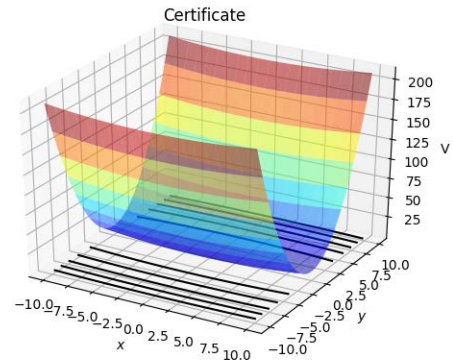

Figure 5: Depiction of the Lyapunov function generated for the $\mathsf{nonpoly}_0$ benchmark.

2-dimensional, third-order model, and with $\mathsf{nonpoly}_2$ - $\mathsf{nonpoly}_3$, which are two 3-dimensional models of order 3 and 4, respectively.

The second set comprises four polynomial benchmarks that admit a global, polynomial Lyapunov function [22, 28]: poly_1 [28] is 2-dimensional model of order 3, poly_2 [22] is a 3-dimensional model of order 3, whereas poly_3 and poly_4 [28] are 2-dimensional models of order 3 and 5 respectively, with no linear terms.

The third set synthesises barrier functions on models from [38]: barr_1, originally presented in [37], has been reported not to admit an LMI-based barrier with degree less than 6; barr_2 [18] encompasses non-polynomial (exponential and trigonometric) terms in the dynamics; barr_3, whose barrier function is depicted in Fig. 6 and whose zero-level set is shown in Fig. 7, is a modification of a benchmark presented in [21] which now uses non-convex domains. Finally, the dynamics in barr_4 [6] has trigonometric functions representing a robotics application, namely the control of the angular velocity of a plane.

Ultimately[2], we challenge FOSSIL on the synthesis of a Lyapunov and barrier functions on models with hybrid dynamics, denoted hy-lyap and hy-barr, which is new for literature on of counterexample-based Lyapunov/barrier synthesis. The hy-lyap dynamics are

$$f(x,y) = \begin{cases} f_1 & \text{if } y < 0, \\ f_2 & \text{otherwise,} \end{cases}$$

where

$$f_1 = \begin{cases} \dot{x} = -x, \\ \dot{y} = -y - 0.5x^3, \end{cases} \qquad f_2 = \begin{cases} \dot{x} = -x, \\ \dot{y} = -y - x^2 - 0.25y^3, \end{cases}$$

with domain $\mathcal{D} = \{x^2 + y^2 \leq 10^2\}$. The hy-barr dynamics are

$$f(x,y) = \begin{cases} f_1 & \text{if } x < 0, \\ f_2 & \text{otherwise,} \end{cases}$$

[2]The bottom three case studies are discussed in Section 5.2.

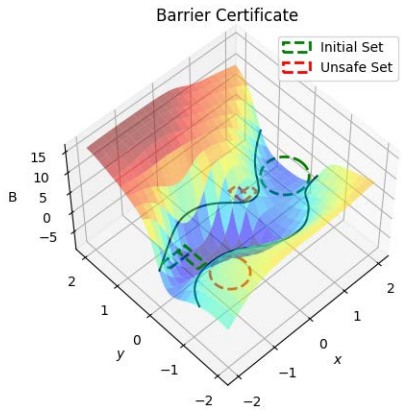

Figure 6: Depiction of the barrier function generated for the $\texttt{barr}_3$ benchmark.

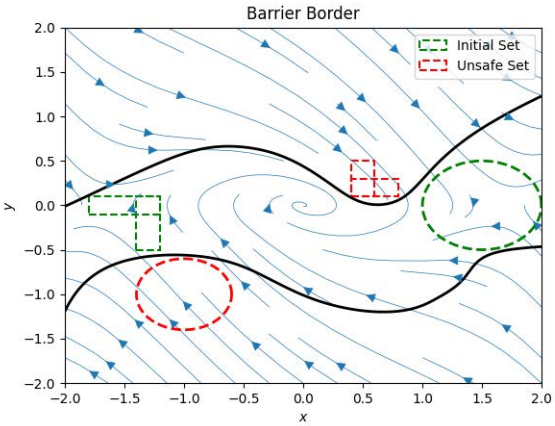

Figure 7: Depiction of the given initial and unsafe sets, and of the barrier function (zero-level set) generated for the $\texttt{barr}_3$ benchmark.

where

$$f_1 = \begin{cases} \dot{x} = y, \\ \dot{y} = -x - 0.5x^3, \end{cases} \qquad f_2 = \begin{cases} \dot{x} = y, \\ \dot{y} = x - 0.25y^2, \end{cases}$$

with overall domains $\mathcal{D} = \{x^2+y^2 \leq 4\}, X_0 = \{(x+1)^2+(y+1)^2 \leq 0.25\}, X_u = \{(x-1)^2 + (y-1)^2 \leq 0.25\}$.

For Lyapunov synthesis, we set for all experiments a spherical domain of radius 10, and as per [11] we restrict the domain of the first four benchmarks to the positive orthant of the state space. The domains of the barrier functions synthesis are instead test-specific and reported in [38].

Let us report that all experiments are run on an unassuming 4-core Dell laptop (2016) with 8GB RAM. Table 1 outlines the results in terms of total computational times, CEGIS iterations, NN setting (activation functions and number of neurons) and the choice of the SMT engine. We denote the NN structure as $h \cdot [\phi_i, \ldots, \phi_j]$: here h represents the number of hidden neurons, which is the same for every layer[3]. Polynomial activations are denoted ϕ_i, where i indicates the order: e.g. ϕ_1 denotes identity activation functions, ϕ_2 quadratic activation functions. By $\phi_{i,j}$ we denote a layer that has a "split activation": namely, by $\phi_{1,2}$ we denote a layer where the first half of the elements have identity activations, whereas the second half has quadratic activation functions. Non-polynomial activation functions are denoted with their name, e.g. `sigmoid`.

We offer a complete set of tests, ranging from high-order polynomials to hybrid systems, many of which cannot be handled by SOS-based techniques due to the presence of non-polynomial terms, of non-convex domains, or simply in view of their complex dynamics. FOSSIL consistently finds certificates for every benchmark in reasonable time: it provides as an additional output a domain of validity, which is a feature that is not provided by SOS-based approaches. We remark that the synthesis procedure quickly converges to correct candidates despite the low number of initial samples (from 500 to

Table 1: Computational benchmarks.
Dim denotes model dimensionality, Iters the number of loops taken by the CEGIS synthesis, Time is the total time for synthesis (seconds), N-Net describes the neural network structure (number of neurons and activation functions - see text), and Verifier denotes the SMT tool utilised.

Benchmark	Dim	Iters	Time	N-Net	Verifier
$\texttt{nonpoly}_0$	2	1	0.22	$2\cdot[\phi_2]$	dReal
$\texttt{nonpoly}_1$	2	1	1.54	$20\cdot[\phi_1, \phi_2]$	Z3
$\texttt{nonpoly}_2$	3	1	1.63	$10\cdot[\phi_1, \phi_2]$	Z3
$\texttt{nonpoly}_3$	3	1	3.87	$4\cdot[\phi_2]$	Z3
$\texttt{poly}_1$	3	1	0.81	$5\cdot[\phi_2]$	dReal
$\texttt{poly}_2$	2	3	19.13	$10\cdot[\phi_2]$	Z3
$\texttt{poly}_3$	2	3	9.61	$10\cdot[\phi_2]$	dReal
$\texttt{poly}_4$	2	2	5.88	$5\cdot[\phi_2]$	dReal
$\texttt{barr}_1$	2	1	11.86	$10\cdot[\phi_1]$	dReal
$\texttt{barr}_2$	2	1	27.32	20·[`softplus`]	dReal
$\texttt{barr}_3$	2	2	65.44	20·[`sigmoid`, `sigmoid`]	dReal
$\texttt{barr}_4$	3	10	30.82	$5\cdot[\phi_{1,2,3}]$	dReal
`hy-lyap`	2	1	2.24	$10\cdot[\phi_2]$	Z3
`hy-barr`	2	1	8.46	$3\cdot[\phi_{1,2}]$	Z3
$\texttt{hi-ord}_4$	4	3	98.94	$20\cdot[\phi_1]$	dReal
$\texttt{hi-ord}_6$	6	1	29.39	$10\cdot[\phi_1]$	dReal
$\texttt{hi-ord}_8$	8	1	41.72	$10\cdot[\phi_1]$	dReal

[3]For simplicity we illustrate the procedure with an equal h, yet FOSSIL supports a different number of hidden neurons per layer. It suffices to set $[h_i \cdot \phi_i, \ldots, h_j \cdot \phi_j]$.

2000 for the proposed benchmarks) and the relatively small size of neural network templates.

Let us conclude with two remarks: notice that the results above are obtained using a predetermined numerical seed that freezes the initialisation of the NN for repeatability purposes; further, notice that we draw benchmarks from the literature on Lyapunov or barrier function synthesis that are low-dimensional (dimension 2, 3). We address these two issues in the following section that focuses on quality evaluation and stress testing.

5.2 Threats to Generality

In this section we demonstrate the stability and quality of our codebase, testing it over models with increasing dimensions, providing repeated analysis of its performance on a benchmark, and discussing software tests that parts of the codebase has undergone.

5.2.1 Scalability. We challenge FOSSIL on three high-order ODEs, respectively of order four, six and eight. The last three rows of Table 1, respectively denoted $\texttt{hi-ord}_4$, $\texttt{hi-ord}_6$ and $\texttt{hi-ord}_8$, report the results along with the experimental settings. We test dynamical models described by a single differential equation: for instance, the eighth-order system evolves according to the ODE

$$x^{(8)} + 20x^{(7)} + 170x^{(6)} + 800x^{(5)} + 2273x^{(4)} + 3980x^{(3)} + 4180x^{(2)} + 2400x^{(1)} + 576 = 0, \tag{17}$$

where we denote the i-th derivative of variable x by $x^{(i)}$. The fourth- and sixth-order models use the last four and six terms of this equation, respectively. We translate the ODE into a state-space model with variables $x_1, \ldots, x_j$, where $j = \{4, 6, 8\}$, respectively. In all three instances, we consider a barrier function problem and we select as spatial domain X an hyper-sphere centred at the origin of radius 4; an initial set X_0 as hyper-sphere[4] centred at $+\mathbf{1}^{[j]}$ of radius 0.25; an unsafe set X_u as an hyper-sphere centred at $-\mathbf{2}^{[j]}$ of radius 0.16. As an example, the eight-dimensional model presents the domains $X = \{x_1^2 + \ldots + x_8^2 \leq 4\}$, $X_0 = \{(x_1-1)^2 + \ldots + (x_8-1)^2 \leq 0.25\}$, $X_u = \{(x_1+2)^2 + \ldots + (x_8+2)^2 \leq 0.16\}$. Whilst in particular the verification engine is challenged from the high dimensionality of the models, the CEGIS procedure returns a valid barrier certificate in only two iterations and within reasonable run times.

For synthesis problems involving many variables (whether in the dynamics or in the candidate functions), the major bottleneck of FOSSIL is represented by the verification engine: formally certifying complex functions critically slows down the engine. Naturally, it is convenient to maintain low-order activation functions in these instances. A possible mitigation of this impairment is represented by a more tailored synthesis approach, in which the complexity of the network is constrained in a process similar to regularisation.

5.2.2 Robustness Analysis. FOSSIL handles randomly generated numbers in a few parts, such as the initial samples set S, the generation of the *cloud* of counterexamples, the initialisation of the neural network: single runs may be favoured or penalised by these random choices. To test the robustness of the underlying CEGIS engine, we run the tool 100 times to synthesise a Lyapunov function for the $\texttt{nonpoly}_0$ benchmark in Eq. (7), over a circular domain $\mathcal{D}$ characterised by a selected radius. For barrier function synthesis we select the $\texttt{barr}_1$ benchmark, whose dynamic evolution is

$$\begin{cases} \dot{x} = y + 2xy, \\ \dot{y} = -x + 2x^2 - y^2, \end{cases} \tag{18}$$

within domain $\mathcal{D} = \{-2 \leq x, y \leq 2\}$ and sets

$$X_0 = \{0 \leq x \leq 1, 1 \leq y \leq 2\}, \quad X_u = \{x + y^2 \leq 0\}.$$

We set a maximum of 10 CEGIS loops and a verification timeout of 60 seconds: if a candidate is not found within this strict limits, the experiment is denoted as a failure. We report the minimum running time, average running time and total number of failures varying the domain radius (for the Lyapunov function instance) and the number of hidden neurons in the NN. In Table 2 and 3 we display the outcomes for the Lyapunov and barrier function instances respectively, using dReal as the verification engine. We report the minimum running time in the first row of every cell, whereas the second row contains the average running time and the number of failures, between parenthesis. As per Table 1 we select a single-layer network with activation ϕ_2 for the model in (7) and a single-layer network with activations ϕ_1 for the model in (18).

We first notice that the tool finds certificates for every combination radius/neurons, witnessing the robustness of our approach. However, a small network reports a large number of failures as the domain widens. As depicted in Fig. 8, where we present the average running times and the failures (marked by a star), we identify a tradeoff, both in terms of average computational time and failures, between the size of the domain $\mathcal{D}$ (a function of the radius) and the dimension of the NN: a smaller one is faster over small domains $\mathcal{D}$, whereas it struggles over larger domains; conversely, a larger NN is more expressive and consistently finds a certificate over large domains.

The barrier function synthesis shows instead zero failures in all runs and a computational time that increases with the number of neurons. We deduce that the chosen network setting is particularly strong: the simplicity of the architecture ensures also enough flexibility to adapt to the particular benchmark.

The computational time is occupied almost exclusively by the learner, which accrues for the 98.8% of the total, whereas the translator, the verifier and the consolidator represent the 0.2%, 0.9% and 0.1% of the total time, respectively. The analysis of the minimum computational times brings us to the next section.

Seed-and-speed. Table 2 has empirically exposed an important evidence: the initialisation of the weight matrices plays a crucial rule for the fast convergence of neural networks. Especially neural architectures with a low number of neurons or of layers, as the ones employed in FOSSIL, are critically dependent on a proper initialisation. For instance, the computational time of $\texttt{nonpoly}_0$ and $\texttt{barr}_1$ with a fixed seed is 0.47 and 26.85 seconds, respectively, whereas the average run for the same configuration is 10.05 and 36.83 seconds, respectively. To exploit this dependency on random seeding in FOSSIL, we implement the *seed-and-speed* feature. This involves calling the whole CEGIS loop on a short timeout, resulting in many initialisations of both the NN and the set S: we rapidly seek to find an instance of the random seeding, where synthesis is

[4]We denote $\mathbf{1}^{[j]}$ the point of a j-dimensional state-space that has all its components equal to 1. For instance, $\mathbf{1}^{[3]}$ is the 3-dimensional point $[1, 1, 1]$. Similarly for $-\mathbf{2}^{[j]}$.

Table 2: Minimum running times (top row of every cell), average running times (bottom row) and number of failures (in parenthesis), out of 100 runs, for Lyapunov synthesis task based on the model in (7).

Radius	Hidden Neurons			
	2	10	50	100
10	0.05 10.05 (10)	0.09 26.95 (16)	0.15 2.10 (0)	1.67 2.90 (2)
20	0.09 18.56 (26)	0.12 0.84 (1)	0.90 1.20 (0)	1.65 2.31 (0)
50	0.52 61.10 (70)	1.09 2.86 (9)	1.45 3.85 (2)	3.90 7.81 (6)
100	0.21 87.70 (83)	5.47 39.95 (52)	5.56 9.95 (18)	12.00 19.31 (17)
200	0.40 119.04 (93)	8.38 72.77 (89)	14.11 47.18 (81)	20.27 54.28 (74)
500	7.83 125.61 (90)	19.26 119.94 (83)	20.39 75.46 (48)	42.59 96.56 (62)

Table 3: Minimum and average running times and number of failures (in parenthesis), out of 25 runs, for barrier synthesis task over model in (18).

Hidden Neurons	2	10	50	100
Min Time	18.11	21.72	61.06	173.01
Avg Time (Fail)	35.46 (0)	33.27 (0)	73.77 (0)	217.68 (0)

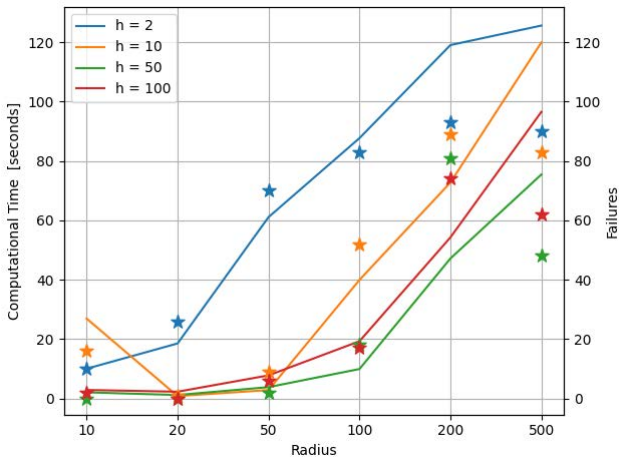

Figure 8: Average running times (straight lines) and number of failures (stars).

quickly successful, thus actively avoiding to incur in long training or verification times. Notice that any resulting function is still correct.

5.2.3 Quality Assurance: Testing. We have assessed the quality and driven the development of our tool with a systematic use of tests. We accompany our tool with a comprehensive test suite not only for reproducing our benchmarks, but also for preventing regression during development. We have followed two driving principles for the design of our tests: the first is that we have ensured their determinism and the second is that we have tested components both integrated in the main loop and in isolation.

Our tool makes use of randomisation in different parts. At the beginning of a CEGIS loop, a number of points in the system are sampled randomly. After the verifier determines that the model violates either the conditions for stability (Lyapunov) or for safety (barriers), the associated counterexample is augmented by the consolidator with points sampled randomly. While randomisation helps the learner generalise efficiently, it makes finding bugs and, generally, reproducing outcomes difficult. For this purpose, we initialise the random number generator with values chosen arbitrarily. Notably, we make our tool deterministic only when testing for bugs. The final performance presented in the previous section result from an aggregation of "truly" random executions.

On top of testing full executions of our tool, we test its two main components, learner and verifier, in isolation. We ensure that the learner satisfies the specification for which, upon termination of the CEGIS loop, every point in the samples set satisfies the conditions for stability or safety: in other words, we ensure that it always reaches a 100% accuracy. Whereas this requirement is strong for common machine learning applications, in the contest of formal synthesis this is necessary for ensuring correctness. Moreover, we test that every outcome of the verifier always correctly satisfies the conditions for stability or safety: this is done with an additional SMT check, giving an extra layer of correctness assurance and preventing regression.

6 CONCLUSIONS

FOSSIL is a new software toolbox for the automated formal synthesis of Lyapunov and barrier functions. It leverages an inductive engine that casts the synthesis problem as an inductive loop between a learner and a verifier, implementing new methods that enhance the communication between the two components. FOSSIL has been tested and challenged on a number of benchmarks, and has shown numerical robustness features.

We plan to implement our on-going theoretical work on future releases of FOSSIL, including: scalability of the approach by learner based on SOS optimisation, synthesis of control Lyapunov/barrier functions on richer (probabilistic and hybrid) models, and the study of multi-equilibria dynamics.

REFERENCES

[1] A. Abate, D. Ahmed, M. Giacobbe, and A. Peruffo. 2021. Formal Synthesis of Lyapunov Neural Networks. *IEEE Control Systems Letters* 5, 3 (2021), 773–778.
[2] Amir Ali Ahmadi, Miroslav Krstic, and Pablo A Parrilo. 2011. A Globally Asymptotically Stable Polynomial Vector Field with No Polynomial Lyapunov Function. In *2011 50th IEEE Conference on Decision and Control and European Control Conference*. IEEE, 7579–7580.
[3] Daniele Ahmed, Andrea Peruffo, and Alessandro Abate. 2020. Automated and Sound Synthesis of Lyapunov Functions with SMT Solvers. In *TACAS (1) (LNCS, Vol. 12078)*. Springer, 97–114.
[4] Daniele Ahmed, Andrea Peruffo, and Alessandro Abate. 2021. Automated Formal Synthesis of Neural Barrier Certificates for Dynamical Models. In *TACAS (LNCS, Vol. 12651)*. Springer, 370–388.
[5] Karl J. Astrom and Richard M. Murray. 2008. *Feedback Systems: An Introduction for Scientists and Engineers.* Princeton University Press, Princeton, NJ, USA.

[6] Andrew J Barry, Anirudha Majumdar, and Russ Tedrake. 2012. Safety Verification of Reactive Controllers for UAV Flight in Cluttered Environments using Barrier Certificates. In *2012 IEEE International Conference on Robotics and Automation*. IEEE, 484–490.

[7] Ya-Chien Chang, Nima Roohi, and Sicun Gao. 2019. Neural Lyapunov Control. In *NeurIPS*. 3240–3249.

[8] Liyun Dai, Ting Gan, Bican Xia, and Naijun Zhan. 2017. Barrier Certificates Revisited. *Journal of Symbolic Computation* 80 (2017), 62–86.

[9] Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: An Efficient SMT Solver. In *TACAS (LNCS, Vol. 4963)*. Springer, 337–340.

[10] Sicun Gao, Soonho Kong, and Edmund M Clarke. 2013. dReal: An SMT Solver for Nonlinear Theories over the Reals. In *International Conference on Automated Deduction*. Springer, 208–214.

[11] Eric Goubault, Jacques-Henri Jourdan, Sylvie Putot, and Sriram Sankaranarayanan. 2014. Finding Non-polynomial Positive Invariants and Lyapunov Functions for Polynomial Systems through Darboux Polynomials. In *ACC*. IEEE, 3571–3578.

[12] Dejan Jovanovic and Leonardo de Moura. 2012. Solving Non-linear Arithmetic. In *IJCAR (LNCS, Vol. 7364)*. Springer, 339–354.

[13] James Kapinski, Jyotirmoy V. Deshmukh, Sriram Sankaranarayanan, and Nikos Aréchiga. 2014. Simulation-guided Lyapunov Analysis for Hybrid Dynamical Systems. In *HSCC*. ACM, 133–142.

[14] Thomas Kluyver, Benjamin Ragan-Kelley, Fernando Pérez, Brian Granger, Matthias Bussonnier, Jonathan Frederic, Kyle Kelley, Jessica Hamrick, Jason Grout, Sylvain Corlay, Paul Ivanov, Damián Avila, Safia Abdalla, and Carol Willing. 2016. Jupyter Notebooks – a publishing format for reproducible computational workflows. In *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, F. Loizides and B. Schmidt (Eds.). IOS Press, 87 – 90.

[15] Hui Kong, Fei He, Xiaoyu Song, William NN Hung, and Ming Gu. 2013. Exponential-condition-based Barrier Certificate Generation for Safety Verification of Hybrid Systems. In *International Conference on Computer Aided Verification*. Springer, 242–257.

[16] Daniel Kroening and Ofer Strichman. 2016. *Decision Procedures - An Algorithmic Point of View*. Springer Verlag.

[17] Benoît Legat, Paulo Tabuada, and Raphaël M Jungers. 2020. Sum-of-Squares Methods for Controlled Invariant Sets with Applications to Model-predictive Control. *Nonlinear Analysis: Hybrid Systems* 36 (2020), 100858.

[18] Jiang Liu, Naijun Zhan, Hengjun Zhao, and Liang Zou. 2015. Abstraction of Elementary Hybrid Systems by Variable Transformation. In *International Symposium on Formal Methods*. Springer, 360–377.

[19] Aaron Meurer, Christopher P. Smith, Mateusz Paprocki, Ondřej Čertík, Sergey B. Kirpichev, Matthew Rocklin, Amit Kumar, Sergiu Ivanov, Jason K. Moore, Sartaj Singh, Thilina Rathnayake, Sean Vig, Brian E. Granger, Richard P. Muller, Francesco Bonazzi, Harsh Gupta, Shivam Vats, Fredrik Johansson, Fabian Pedregosa, Matthew J. Curry, Andy R. Terrel, Štěpán Roučka, Ashutosh Saboo, Isuru Fernando, Sumith Kulal, Robert Cimrman, and Anthony Scopatz. 2017. SymPy: symbolic computing in Python. *PeerJ Computer Science* 3 (Jan. 2017), e103. https://doi.org/10.7717/peerj-cs.103

[20] Antonis Papachristodoulou, James Anderson, Giorgio Valmorbida, Stephen Prajna, Pete Seiler, and Pablo Parrilo. 2018. SOSTOOLS Version 3.03. Sum of Squares Optimization Toolbox for MATLAB.

[21] Stephen Prajna. 2006. Barrier Certificates for Nonlinear Model Validation. *Automatica* 42, 1 (2006), 117–126.

[22] Stephen Prajna, Antonis Papachristodoulou, and Pablo A. Parrilo. 2004. SOSTOOLS: Sum of squares Optimization Toolbox for MATLAB–User's Guide. *Control and Dynamical Systems, California Institute of Technology, Pasadena, CA* 91125 (2004).

[23] Stefan Ratschan and Zhikun She. 2010. Providing a Basin of Attraction to a Target Region of Polynomial Systems by Computation of Lyapunov-Like Functions. *SIAM J. Control and Optimization* (2010).

[24] Hadi Ravanbakhsh and Sriram Sankaranarayanan. 2015. Counter-Example Guided Synthesis of Control Lyapunov Functions For Switched Systems. In *IEEE Control and Decision Conference (CDC)*. 4232–4239.

[25] Hadi Ravanbakhsh and Sriram Sankaranarayanan. 2016. Robust Controller Synthesis of Switched Systems Using Counterexample Guided Framework. In *ACM/IEEE Conference on Embedded Software (EMSOFT)*. 8:1–8:10.

[26] Hadi Ravanbakhsh and Sriram Sankaranarayanan. 2018. Learning Control Lyapunov Functions from Counterexamples and Demonstrations. *Autonomous Robots* (2018), 1–33.

[27] Spencer M. Richards, Felix Berkenkamp, and Andreas Krause. 2018. The Lyapunov Neural Network: Adaptive Stability Certification for Safe Learning of Dynamical Systems. In *CoRL (Proceedings of Machine Learning Research, Vol. 87)*. PMLR, 466–476.

[28] Sriram Sankaranarayanan, Xin Chen, and Erika Abraham. 2013. Lyapunov Function Synthesis using Handelman Representations. *IFAC Proceedings Volumes* 46, 23 (2013), 576–581.

[29] Mohamed Amin Ben Sassi, Sriram Sankaranarayanan, Xin Chen, and Erika Ábrahám. 2016. Linear Relaxations of Polynomial Positivity for Polynomial Lyapunov Function Synthesis. *IMA J. Math. Control & Information* 33, 3 (2016), 723–756.

[30] Shankar Sastry. 1999. *Nonlinear Systems: Analysis, Stability and Control*. Springer Verlag.

[31] Zhikun She, Haoyang Li, Bai Xue, Zhiming Zheng, and Bican Xia. 2013. Discovering Polynomial Lyapunov Functions for Continuous Dynamical Systems. *Journal of Symbolic Computation* 58 (2013), 41–63.

[32] Zhikun She, Bican Xia, Rong Xiao, and Zhiming Zheng. 2009. A Semi-algebraic Approach for Asymptotic Stability Analysis. *Nonlinear Analysis: Hybrid Systems* 3, 4 (2009), 588–596.

[33] Christoffer Sloth, George J Pappas, and Rafael Wisniewski. 2012. Compositional Safety Analysis using Barrier Certificates. In *Proceedings of the 15th ACM international conference on Hybrid Systems: Computation and Control*. 15–24.

[34] Andrew Sogokon, Khalil Ghorbal, Yong Kiam Tan, and André Platzer. 2018. Vector Barrier Certificates and Comparison Systems. In *International Symposium on Formal Methods*. Springer, 418–437.

[35] Armando Solar-Lezama, Liviu Tancau, Rastislav Bodik, Sanjit Seshia, and Vijay Saraswat. 2006. Combinatorial sketching for finite programs. In *Proceedings of the 12th international conference on Architectural support for programming languages and operating systems*. 404–415.

[36] Cees F. Verdier and Manuel Mazo Jr. 2018. Formal Synthesis of Analytic Controllers for Sampled-Data Systems via Genetic Programming. In *CDC*. IEEE, 4896–4901.

[37] Xia Zeng, Wang Lin, Zhengfeng Yang, Xin Chen, and Lilei Wang. 2016. Darboux-type Barrier Certificates for Safety Verification of Nonlinear Hybrid Systems. In *Proceedings of the 13th International Conference on Embedded Software*. 1–10.

[38] Hengjun Zhao, Xia Zeng, Taolue Chen, and Zhiming Liu. 2020. Synthesizing Barrier Certificates Using Neural Networks. In *Proceedings of the 23rd International Conference on Hybrid Systems: Computation and Control* (Sydney, New South Wales, Australia) *(HSCC '20)*. Association for Computing Machinery, New York, NY, USA, Article 25, 11 pages. https://doi.org/10.1145/3365365.3382222